

Padrões de Projeto (Singleton)

QXD0068 - Reuso de Software

Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução
- Padrão Singleton
- Problema
- Solução
- Analogia do Mundo Real
- Estrutura do Padrão
- Exemplo em Kotlin e Python
- Aplicabilidade
- Prós e Contras
- Implementação

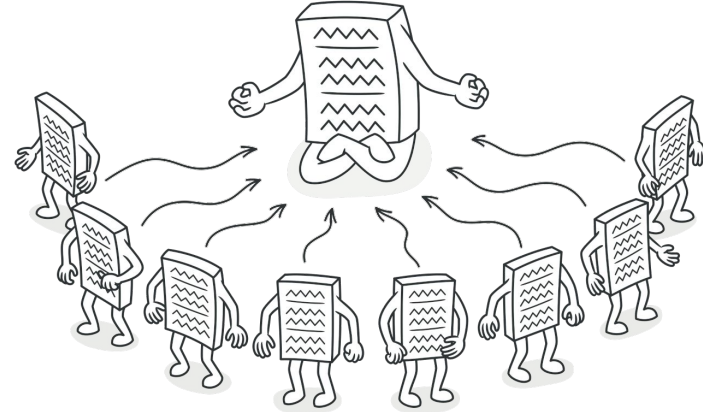


Singleton

Introdução

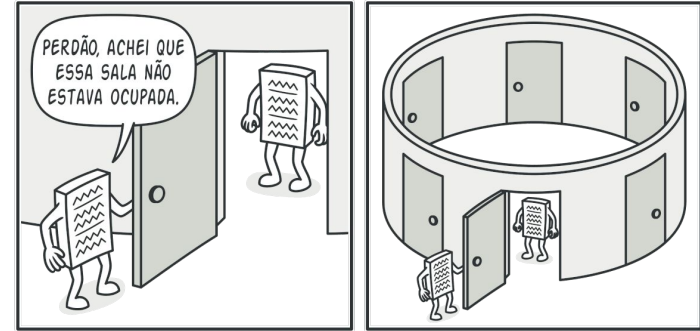
- O Singleton é um padrão de projeto criacional que permite a você garantir que uma classe tenha apenas uma instância, enquanto provê um ponto de acesso global para essa instância.
- O termo vem do significado em inglês para um conjunto (entidade matemática) que contenha apenas um elemento.

Singleton
- instance : Singleton = null
+ getInstance() : Singleton
- Singleton() : void



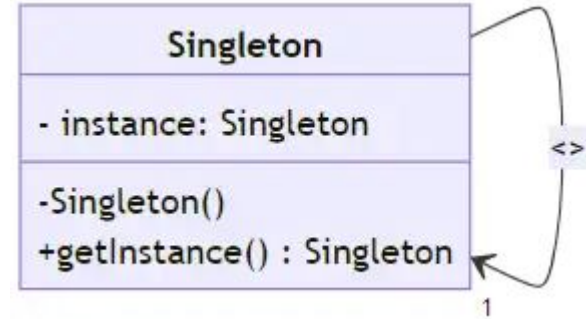
Problema

- Em algumas situações, uma aplicação precisa garantir que exista apenas uma instância de determinada classe, principalmente quando ela controla um recurso compartilhado, como uma conexão com banco de dados, um arquivo ou uma configuração global.
- Além disso, diferentes partes do sistema precisam acessar essa mesma instância de forma segura. O uso de variáveis globais pode atender parcialmente essa necessidade, mas permite que o objeto seja substituído ou alterado indevidamente.
- Portanto, o problema consiste em controlar a criação de uma única instância e disponibilizar um ponto de acesso global e controlado a ela.



Solução

- Todas as implementações do Singleton tem esses dois passos em comum:
 - Fazer o construtor padrão privado, para prevenir que outros objetos usem o operador new com a classe singleton.
 - Criar um método estático de criação que age como um construtor. Esse método chama o construtor privado por debaixo dos panos para criar um objeto e o salva em um campo estático. Todas as chamadas seguintes para esse método retornam o objeto em cache.
- Se o seu código tem acesso à classe singleton, então ele será capaz de chamar o método estático da singleton. Então sempre que aquele método é chamado, o mesmo objeto é retornado.



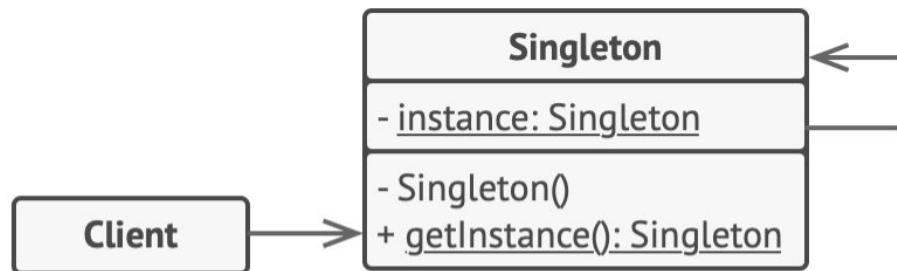
Analogia do Mundo Real

- Um governo pode ser usado como uma analogia para o padrão Singleton. Em um determinado país, existe um único governo oficial em exercício, responsável por representar e administrar o Estado.
- Mesmo que diferentes pessoas ocupem seus cargos ao longo do tempo, o conceito de “Governo do país” continua sendo uma referência única. Assim, quando cidadãos, empresas ou instituições precisam interagir com o governo, todos recorrem ao mesmo ponto de referência oficial, em vez de criarem diferentes governos independentes.
- Da mesma forma, o Singleton garante que exista uma única instância de uma classe e fornece um ponto global de acesso a essa instância.



Estrutura do Padrão

- A estrutura do padrão Singleton é simples e gira em torno de uma única classe:
 - **Singleton:** classe cuja instância deve ser única.
 - **instance:** atributo privado e estático que armazena a única instância da classe.
 - **Construtor privado:** impede que outras classes criem objetos usando new.
 - **getInstance():** método público e estático responsável por criar a instância na primeira chamada e retornar sempre a mesma nas chamadas seguintes.
 - **Métodos de negócio:** funcionalidades normais oferecidas pela classe Singleton.
 - **Cliente:** não instancia diretamente a classe; acessa o objeto por meio de getInstance().



1 A classe **Singleton** declara o método estático `getInstance` que retorna a mesma instância de sua própria classe.

O construtor da singleton deve ser escondido do código cliente. Chamando o método `getInstance` deve ser o único modo de obter o objeto singleton.

```

if (instance == null) {
    // Atenção: se você está criando uma
    // aplicação com apoio multithreading,
    // você deve colocar um thread lock aqui.
    instance = new Singleton()
}
return instance
  
```

```
object Configuracao {

    var tema: String = "Claro"

    fun mostrarConfiguracao() {
        println("Tema atual: $tema")
    }
}

fun main() {

    val config1 = Configuracao
    val config2 = Configuracao

    config1.tema = "Escuro"

    config2.mostrarConfiguracao()

    println(config1 === config2)
}
```

Exemplo em Kotlin

```
class SingletonMeta(type):
    """
    A classe Singleton pode ser implementada de diferentes formas em Python.
    Alguns métodos possíveis incluem: classe base, decorator e metaclasses.
    Utilizaremos a metaclasses porque ela é mais adequada para esse propósito.
    """

    _instances = {}

    def __call__(cls, *args, **kwargs):
        """
        Possíveis alterações nos valores dos argumentos de `__init__`
        não afetam a instância retornada.
        """
        if cls not in cls._instances:
            instance = super().__call__(*args, **kwargs)
            cls._instances[cls] = instance
        return cls._instances[cls]

class Singleton(metaclass=SingletonMeta):
    def some_business_logic(self):
        """
        Por fim, qualquer Singleton deve definir alguma lógica de negócio
        que possa ser executada em sua instância.
        """

        # ...
```

Exemplo em Python

```
if __name__ == "__main__":
    # Código cliente.

    s1 = Singleton()
    s2 = Singleton()

    if id(s1) == id(s2):
        print("O Singleton funciona: ambas as variáveis contêm a mesma instância.")
    else:
        print("O Singleton falhou: as variáveis contêm instâncias diferentes.")
```

Aplicabilidade

- **Use o Singleton quando:**
 - Quando deve existir apenas uma instância de uma classe.
 - Quando essa instância precisa ser compartilhada por diferentes partes do sistema.
 - Quando é necessário controlar o acesso a recursos globais, como banco de dados, configuração ou logs.
 - Quando se deseja evitar o uso direto de variáveis globais.
 - Quando a criação do objeto deve ocorrer por um ponto de acesso controlado, como getInstance().
 - Quando é importante impedir que outras partes do sistema substituam ou recriem a instância existente.

Prós e Contras

- **Vantagens**

- Garante uma única instância.
- Fornece acesso global controlado.
- Permite inicialização sob demanda.

- **Desvantagens**

- Viola o princípio de responsabilidade única.
- Pode mascarar problemas de design.
- Exige cuidado em ambientes multithread.
- Pode dificultar testes unitários.



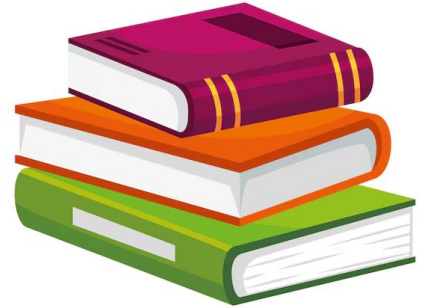
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

